

Pré-Processamento de Dados de PLN

Fabício Galende Marques de Carvalho

OBS: Esse texto foi produzido com o proposito de servir como um guia de estudo da disciplina Processamento de Linguagem Natural, ministrada por Fabício Galende Marques de Carvalho, seja em cursos de graduação, pós-graduação ou extensão universitária. É permitida sua livre utilização e reprodução por parte dos alunos matriculados e que tenham concluído cursos dessa área que tenham sido integralmente ministrados pelo autor do material. Qualquer outro uso ou redistribuição deve ser feito somente mediante autorização, por escrito, fornecida pelo autor ao participante. Caso o conteúdo desse trabalho seja usado para subsidiar outras publicações científicas ou tecnológicas, esse deve ser citado da seguinte forma:

Carvalho, F. G. M. Notas de aula do curso Processamento de Linguagem Natural. São José dos Campos, 2025.

Resumo

Esse texto trata da etapa de pré-processamento de dados aplicável a uma *pipeline* de PLN. São descritas em linhas gerais atividades que vão desde o processo de extração dos dados, a partir de determinada fonte, até sua disponibilização para a etapa seguinte, correspondente à extração de características e/ou representação do texto em formato apropriado ao processamento computacional.

1 Introdução

Conforme dito no texto introdutório da disciplina de PLN, as fontes de dados de linguagem natural possuem natureza diversificada. Por exemplo, logs de interações via chat, e-mails, páginas da Internet, etc. Dessa forma, é esperado que também seja diversificado o formato no qual o texto seja disponibilizado. Tal diversidade deve ser primeiramente tratada e o texto padronizado antes que se possa de fato manipular os dados linguísticos e extrair informações e conhecimentos relevantes. A etapa que lida com tal padronização é justamente a etapa de **pré-processamento**, que sucede imediatamente a etapa de coleta de dados linguísticos.

A etapa de pré-processamento, também conhecida como etapa de **normalização** ou *text wrangling* é a primeira etapa de uma pipeline completa de PLN. Nessa etapa, informações irrelevantes contidas no texto são removidas, são efetuadas conversões de caixa (i.e, normalização de caixa), palavras são reduzidas às suas formas básicas, são efetuadas correções ortográficas, são efetuadas expansões de abreviaturas e siglas, etc.

Nesse texto são discutidas e exemplificadas cada uma dessas etapas de modo a habilitar o leitor a desenvolver sua própria pipeline de pré-processamento ou a usar componentes prontamente disponíveis

em bibliotecas tais como o Natural Language Toolkit (NLTK) e o Spacy.

2 Extração de Dados

A etapa de coleta de dados linguísticos, conhecida como **extração de dados**, as vezes é considerada como a primeira etapa de pré-processamento. Entretanto, quando a extração apenas recupera os dados sem efetuar qualquer tratamento, esta pode ser considerada como uma atividade prévia ao pré-processamento em si. Para fins didáticos nesse texto, consideramos que a extração faz parte do pré-processamento.

Nessa etapa, a entrada para os processos pode ser uma informação de consulta a um banco de dados, uma URL na Internet, uma entrada proveniente do usuário, etc. Ou seja, depende de como os dados a serem usados pela pipeline estão sendo armazenados ou como seu acesso é disponibilizado. Esquemáticamente tem-se:

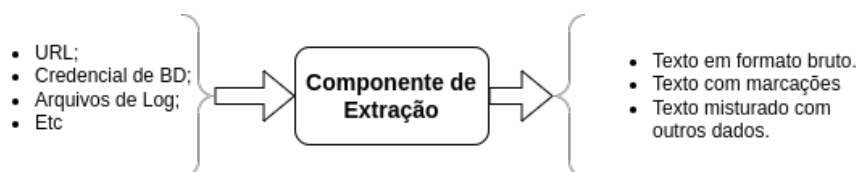


Figura 1: Etapa de extração de dados.

Note-se que a Figura 1 ilustra o fato de que a etapa de extração dos dados recebe como entrada um metadado (e.g., uma URL, um caminho etc.) ou um dado de acesso (e.g., uma credencial) e retorna o texto em formato bruto (*raw text*) ou texto contendo marcações e/ou texto misturado com outros dados.

A Figura 2 mostra um exemplo típico de saída do processo de extração a partir de uma entrada do tipo URL (nesse caso, <https://www.gutenberg.org/cache/epub/53101/pg53101.txt>).

```
Raw string
'\uffeffThe Project Gutenberg eBook of A Mao e A Luva\r\n    \r\nThis ebook is for the
use of anyone anywhere in the United States and\r\nmost other parts of the world at no
cost and with almost no restrictions\r\nwhatsoever. You may copy it, give it away or r
e-use it under the terms\r\nof the Project Gutenberg License included with this ebook
or online\r\nat www.gutenberg.org. If you are not located in the United States,\r\nyou
will have to check the laws of the country where you are located\r\nbefore using this
eBoo'
```

```
Formatted string
The Project Gutenberg eBook of A Mao e A Luva

This ebook is for the use of anyone anywhere in the United States and
most other parts of the world at no cost and with almost no restrictions
whatsoever. You may copy it, give it away or re-use it under the terms
of the Project Gutenberg License included with this ebook or online
at www.gutenberg.org. If you are not located in the United States,
you will have to check the laws of the country where you are located
before using this eBook
```

Figura 2: Etapa de extração de dados.

Durante a etapa de extração, é importante que o desenvolvedor considere os detalhes relacionados a como o texto está sendo armazenado e representado. Mais especificamente, aspectos de mapeamento de **representação e codificação** (i.e., *encoding*) devem ser tratados com atenção.

Com relação a representação, a que é mais comum hoje em dia é a baseada no padrão **Unicode**. Esse padrão engloba quase que a totalidade dos símbolos utilizados, incluindo todos aqueles que são usados na língua escrita (e.g., texto simples em vários idiomas, símbolos matemáticos, etc.). No padrão unicode, cada símbolo possui uma identificação numérica única (i.e., pontos de código de U+0000 até U+10FFFF).

Além do mapeamento para um identificador abstrato único, os dados de texto podem ser codificados de diferentes formas. Uma das formas que é muito utilizada hoje em dia é a baseada no padrão UTF-8 (*Unicode Transformation Format*). Segundo esse encoding, cada símbolo pode ocupar de 8 a 32 bits (ou seja, de 1 a 4 bytes). O que determina a quantidade de bytes utilizados na leitura são padrões de bits específicos para o byte inicial (0xxxxxxx para 1 byte, 110xxxxx para 2 bytes, 1110xxxx para 3 bytes e 11110xxx para 4 bytes).

Além da questão da codificação, um outro aspecto importante que deve ser considerado na etapa de extração são os mecanismos de segurança e construção de sites utilizados na Internet. Muitas vezes portais de venda de produtos, sites de notícias, etc., constroem suas soluções utilizando mecanismos de segurança que impedem que requisições sejam feitas a partir de componentes de software (e.g., *requests* do Python) ou então criam mecanismos de carregamento dinâmico de conteúdo, via scripts, que dificultam o processo de extração de dados. Nesses casos, dependendo do propósito da extração, deve-se contactar os responsáveis pelos sistemas ou utilizar mecanismos que possibilitem a adequada execução da extração (e.g., uso de web drivers tais como Selenium, escrita de headers que emulam navegadores nas requisições, etc.).

3 Remoção de Marcação

A remoção de marcação é a atividade da pipeline de pré-processamento que tipicamente é usada quando se efetua raspagem de dados.

Nesse caso, a entrada desse processo são dados de texto contendo marcação (e.g., HTML, XML, LaTeX, etc.) que carrega potencialmente referências ou outros conteúdos não textuais que não são de interesse da aplicação. Essa atividade é esquematizada na Figura 3



Figura 3: Remoção de marcação de texto.

A remoção de linguagem de marcação pode ser efetuada utilizando-se componentes de software específicos contendo analisadores de linguagem (do inglês, *language parsers*).

A Figura 4 ilustra um texto bruto contendo marcação HTML. Já a Figura 5 ilustra a saída utilizando-se uma biblioteca, da linguagem Python, denominada Beautiful Soup.

Para o exemplo mostrado, a limpeza das marcações foi efetuada simplesmente filtrando-se os elementos correspondentes a parágrafos. Note-se que esse tipo de operação funcionará somente nos casos em que

```
"Request response models a simple http request raw response. "
request_response_text = '''
<body>
<p>The product is good!</p>
<p>It was delivered on time. </p>
<p>Definetely I will buy it again! </p>
<script> var x = 1; </script>
<iframe> contents... </iframe>
<p>Thanks folks! </p>
</body>
'''
```

Figura 4: Texto bruto contendo marcação.

```
['The product is good!', 'It was delivered on time. ',
'Definetely I will buy it again! ', 'Thanks folks! ']
```

Figura 5: Texto após remoção de marcação HTML.

o documento HTML tiver sido bem formado, ou seja, que as tags apropriadas tenham sido usadas para estruturar o conteúdo.

Para os casos em que haja, por exemplo, conteúdos textuais fora de parágrafos ou então em elementos sem valor semântico (e.g., utilizando *divs*) operações de desempacotamento (do inglês, *unwrapping*) ou de destruição de elementos sem conteúdo relevante (e.g., *decomposing*) poderão ser requeridas.

Uma outra maneira interessante e útil de se efetuar a limpeza de marcações é através da utilização das **expressões regulares**.

Uma expressão regular (do inglês regex - *regular expression*) é uma construção que faz uso de cadeias de caracteres (i.e., *strings*) para buscar, validar ou manipular texto. Tais expressões seguem princípios parecidos mas são específicas de cada linguagem de programação. Em essência, expressões regulares incluem caracteres, cadeias de caracteres ou **metacaracteres**. Enquanto caracteres e cadeias de caracteres dizem respeito ao conteúdo textual que será buscado ou manipulado, os metacaracteres informam como tal busca ou manipulação deve ocorrer.

A Tabela 3 ilustra alguns metacaracteres comumente utilizados em regex válido para a linguagem Python.

Metacaractere	Significado
.	Qualquer caractere, exceto nova linha
	Operação de OU entre strings
*	Zero ou mais caracteres igual ao elemento anterior
?	Captura não gulosa ou caractere opcional
\b	Limite (boundary) de uma sequência de caracteres
()	Grupo de caracteres

Tabela 1: Exemplos de metacaracteres utilizados em regex do Python

A Figura 6 ilustra um trecho que simula uma resposta à requisição HTTP. Já a Figura 7 mostra o resultado da extração através da utilização de uma expressão regular do tipo `regexfilter = r' < p >`

(.*?) < /p >', ou seja, que faz uso de um grupo de captura, distintos caracteres potencialmente repetidos e captura de caracteres não gulosa.

```
request_response_text_3 = '''
<body>
<p>The product is good!</p>
<p>It was delivered on time. </p>
<p>Definetely I will buy it again! </p>
<script> var x = 1; </script>
<div>This content is also poorly structured. </div>
<iframe> This content is not relevant. </iframe>
<p>Thanks folks! </p>
</body>
'''
```

Figura 6: Resposta de texto simulada HTTP.

```
['The product is good!', 'It was delivered on time. ', 'Definetely  
I will buy it again! ', 'Thanks folks! ']
```

Figura 7: Conteúdo extraído com expressão regular.

4 Tokenização

Em PLN, um *token* é uma sequência de caracteres, obtidas a partir do texto original, que é tratada como uma unidade indivisível para uma determinada finalidade de processamento.

Tokens podem ser de diferentes tipos incluindo:

- Pontuação (e.g., !)
- Números (e.g., 2025)
- Símbolos ou caracteres especiais (e.g., #)
- Caracteres simples (e.g., "d")
- Subpalavras (e.g., "des", de "desenvolvimento")
- Palavras (e.g., "desenvolvimento")

Cada tipo de token possui uma finalidade específica a qual pode limitar o sistema de PLN a ser desenvolvido.

O processo de obtenção de tokens a partir do texto original é denominado de *tokenização*.

Nessa seção, discutem-se dois tipos específicos de tokenização: a tokenização por sentença e (ou frases) e a tokenização por palavra.

4.1 Tokenização por Sentenças

Constituem um primeiro nível de quebra de um determinado documento. Um documento é constituído, em geral, por um ou vários parágrafos que, por sua vez, podem ser divididos em uma ou mais frases (sentenças).

Tipicamente, a divisão de um documento em um token de sentença tem como objetivo atrelar um certo significado a tal sentença. Pode-se, por exemplo, processar uma sentença de modo a qualificá-la como uma pergunta ou comando a um assistente virtual.

Algoritmos comuns de tokenização por frases incluem: busca por delimitadores específicos (ex. Pontos finais, ponto e vírgula, etc.), modelos de tokenização pré-treinados e tokenização através de expressão regular (regex).

A Figura 8 ilustra esquematicamente o processo de tokenização por sentenças.

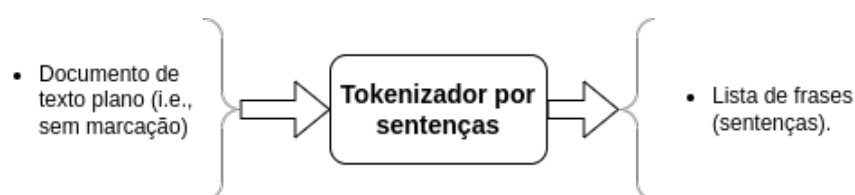


Figura 8: Etapa de tokenização por sentenças.

Em geral, bibliotecas de PLN tais como o NLTK ou Spacy possuem tokenizadores de sentença pré-treinados. Cabe ressaltar, entretanto, que tipicamente esses tokenizadores não fornecem saídas apropriadas a aplicações para produção, sendo necessárias etapas adicionais de tokenização das sentenças de saída ou então o ajuste fino dos modelos utilizados para tokenização.

A Figura 9 mostra um exemplo de entrada e saída de um tokenizador padrão de sentença do NLTK.

```
Raw text input:
Hoje de manhã Pedro acordou com muita fome. Levantou,
escovou os dentes, lavou o rosto e foi para a cozinha.
Abriu a geladeira, pegou uma jarra de suco e três ovos.
Tomou um copo d'água. Fritou os ovos. Sentou na mesa e saboreou seu café da
manhã...

Tokenizer output:
Total sentence tokens: 6
Token # 1 :
Hoje de manhã Pedro acordou com muita fome.
Token # 2 : Levantou,
escovou os dentes, lavou o rosto e foi para a cozinha.
Token # 3 : Abriu a geladeira, pegou uma jarra de suco e três ovos.
Token # 4 : Tomou um copo d'água.
Token # 5 : Fritou os ovos.
Token # 6 : Sentou na mesa e saboreou seu café da
manhã...
```

Figura 9: Entrada e saída de um tokenizador de sentença.

4.2 Tokenização por Palavras

Esse tipo de tokenização consiste na quebra de uma frase em palavras que a constituem. A entrada para esse processo em geral é uma sentença, ou conjunto de sentenças, e a saída é uma lista de palavras

pertencentes a cada sentença, preservando a correspondência entre palavra e sentença. Esse processo é ilustrado esquematicamente na Figura 10.



Figura 10: Processo de tokenização por palavras.

A Figura 11 ilustra um exemplo de entrada e saída do processo de tokenização por palavras utilizando o tokenizador padrão disponível no pacote NLTK do Python.

```

Sentence Tokens (input): ['Hoje de manhã Pedro acordou com muita fome.',
'Levantou, escovou os dentes, lavou o rosto e foi para a cozinha.', 'Abriu
a geladeira, pegou uma jarra de suco e três ovos.', "Tomou um copo d'águ
a.", 'Fritou os ovos.', 'Sentou na mesa e saboreou seu café da manhã...']

Word tokens: [['Hoje', 'de', 'manhã', 'Pedro', 'acordou', 'com', 'muita',
'fome', '.'], ['Levantou', ',', 'escovou', 'os', 'dentes', ',', 'lavou',
'o', 'rosto', 'e', 'foi', 'para', 'a', 'cozinha', '.'], ['Abriu', 'a', 'gel
adeira', ',', 'pegou', 'uma', 'jarra', 'de', 'suco', 'e', 'três', 'ovos',
'.'], ['Tomou', 'um', 'copo', "d'água", '.'], ['Fritou', 'os', 'ovos',
'.'], ['Sentou', 'na', 'mesa', 'e', 'saboreou', 'seu', 'café', 'da', 'manh
ã', '...']]

First word token extracted from first sentence: Hoje
  
```

Figura 11: Exemplo de entrada e saída da tokenização por palavras.

Conforme pode ser notado, a saída do tokenizador padrão do pacote NLTK obteve, além das próprias palavras, tokens correspondentes a marcação de pontuação que, obviamente, não correspondem à saída desejada. Nesse caso pode-se facilmente remover os tokens que não são palavras utilizando-se recursos tais como expressões regulares, tais como vistos anteriormente.

5 Remoção ou Alteração de Caracteres especiais

Caracteres ou símbolos especiais podem carregar ou não significado dependendo da finalidade do sistema de PLN. Exemplos de caracteres especiais podem incluir acentos, sinais de pontuação, símbolos monetários, emojis, etc.

A entrada e a saída do processo de alteração ou remoção de caracteres especiais são ilustrados na Figura 12.

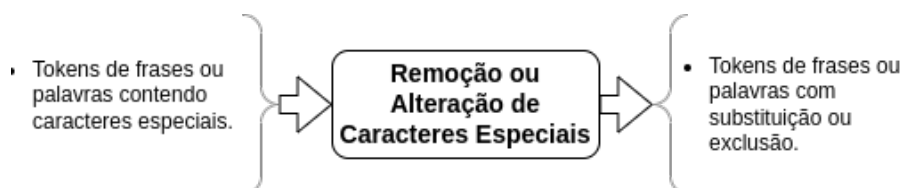


Figura 12: Alteração ou remoção de caracteres especiais.

Para o caso de acentos, por exemplo, na língua portuguesa trata-se de um símbolo que pode diferenciar a classe gramatical da palavra (e.g., é é um verbo enquanto que e é uma conjunção). Nesse caso, a remoção só pode ser efetuada caso a acentuação não altere a classe gramatical e seja útil, por exemplo, para evitar que formas escritas de maneira incorreta (e.g., tais como palavras não acentuadas) sejam tratadas como palavras diferentes daquelas que foram escritas corretamente. Por outro lado, caracteres tais como o trema (e.g., ü) foram extintos da língua portuguesa e devem ser mapeados para seus correspondentes sem o seu uso.

Exemplos:

- Ele é um bom estudante. (é – verbo)
- Ele vai à aula e ele também dorme (e – conjunção coordenativa aditiva)
- Ele é uma pessoa tranqüila ("tranqüila" deve ser mapeada para "tranquila")

A Figura 13 ilustra um exemplo de entrada e saída de substituição de caracteres especiais efetuada em dois níveis de processamento utilizando expressões regulares.

```
Original text (input):  Fabrício mostrou-se uma pessoa muito tranqüila durante a reunião!
First level replacement: Fabrício mostrou-se uma pessoa muito tranquila durante a reunião!
Second level replacement: Fabrício mostrou-se uma pessoa muito tranquila durante a reunião!
```

Figura 13: Entrada e saída do processo de substituição de caracteres especiais.

Para o caso de emojis, uma alternativa é a utilização de uma tabela de hash que mapeie os seus códigos para expressões linguísticas comumente aceitas como equivalentes (e.g., 😊 é mapeado para "sorriso").

Em idiomas tais como o inglês, outro lado, palavras não são acentuadas e, portanto, podem significar erros ortográficos que devem ser devidamente tratados.

6 Remoção de Palavras de Parada (Stop words removal)

Palavras de parada (stop words) são palavras que são muito frequentes em uma linguagem e que podem não agregar muito significado, prejudicando assim o processamento subsequente. A Figura 14 ilustra o processo de remoção de palavras de parada.

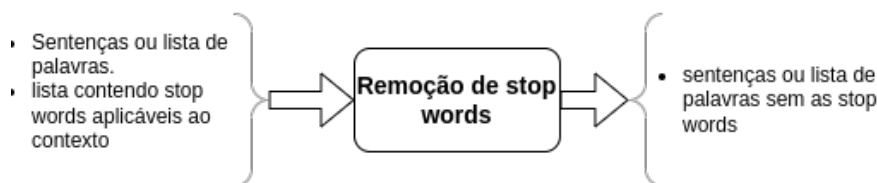


Figura 14: Processo de remoção de palavras de parada.

Exemplos:

Na língua portuguesa, tem-se artigos, preposições, etc., tais como a, as, o, os, de, do, aquele, aquela.

Observação: Algumas vezes, remover stop words pode alterar significativamente o sentido de uma frase e reduzi-la a outra totalmente diferente.

A Tabela 2 ilustra como a remoção indevida de palavras de parada pode fazer com que frases que possuem sentido totalmente diferente sejam reduzidas a um mesmo conjunto de palavras que potencialmente expressa o mesmo sentido. Para esse caso, foram consideradas como stop words $sw = eu, não, do$:

Frase original	Frase após remoção de stop word
Eu não gostei do computador	gostei computador
Eu gostei do computador	gostei computador

Tabela 2: Exemplo de remoção indevida de palavra de parada: não

Dessa forma, pode-se concluir que a remoção ou não de stop words e quais stop words devem ser consideradas depende significativamente do contexto da aplicação de pln bem como da análise criteriosa do corpus ou corpora de texto utilizados no treinamento dos modelos de PLN/IA.

A Figura 15 ilustra uma lista de stop words disponível no pacote NLTK e a execução do exemplo de redução a uma mesma lista de duas frases contendo significados totalmente diferentes.

```
Portuguese stop words: {'estivéramos', 'estiverem', 'fossem', 'teriam', 'isto', 'nossos', 'tenho', 'a
quela', 'das', 'forem', 'teremos', 'tivéramos', 'estiver', 'já', 'estes', 'aquilo', 'estavam', 'fôssem
os', 'lhe', 'pelos', 'tem', 'tere', 'fomos', 'um', 'aquelas', 'com', 'nos', 'tive', 'elas', 'esta',
'de', 'esteja', 'houvermos', 'tenham', 'esses', 'dela', 'aquele', 'mais', 'dos', 'isso', 'terão', 'tiv
eram', 'é', 'hajamos', 'tuas', 'houvessem', 'hão', 'pela', 'houveriam', 'estar', 'ela', 'quem', 'eu',
'também', 'esse', 'delas', 'tivera', 'você', 'houve', 'mesmo', 'qual', 'uma', 'eram', 'num', 'se', 'ho
uvéramos', 'ele', 'às', 'tenhamos', 'estas', 'estivéssemos', 'foram', 'teve', 'vocês', 'meus', 'haja
m', 'tiverem', 'meu', 'minhas', 'teríamos', 'estiveram', 'estamos', 'e', 'nossas', 'estivera', 'sua',
'haja', 'depois', 'fora', 'dele', 'esteve', 'teu', 'houveremos', 'tiver', 'vos', 'tenhamos', 'serei',
'estivemos', 'há', 'houvemos', 'temos', 'suas', 'do', 'houverá', 'teus', 'houverei', 'até', 'estivermo
s', 'seríamos', 'seria', 'houverem', 'hei', 'estivessem', 'no', 'o', 'sejam', 'estive', 'houver', 'd
a', 'houverão', 'houveria', 'na', 'sejamos', 'pelo', 'for', 'essas', 'este', 'tém', 'sem', 'seja', 'ti
vesse', 'nem', 'à', 'eles', 'está', 'houveram', 'por', 'essa', 'seus', 'em', 'quando', 'estivesse', 'e
ra', 'estávamos', 'como', 'haver', 'foi', 'te', 'tinha', 'muito', 'houvera', 'me', 'seremos', 'havemo
s', 'tenha', 'fosse', 'deles', 'as', 'éramos', 'estejamos', 'pelas', 'aos', 'minha', 'ou', 'será', 'ti
véssemos', 'nas', 'que', 'tua', 'estava', 'aqueles', 'tivessem', 'ao', 'terá', 'estejam', 'nosso', 'nã
o', 'formos', 'nossa', 'teria', 'estão', 'lhes', 'seriam', 'tenham', 'houveríamos', 'numa', 'somos',
'nós', 'seu', 'são', 'houvéssemos', 'os', 'serão', 'tivermos', 'para', 'fui', 'houvesse', 'sou', 'só',
'a', 'entre', 'tu', 'fôramos', 'mas', 'ser', 'tivemos', 'estou'}
Original negative review: eu não gostei do computador
Words list without stop words: ['gostei', 'computador']
Original positive review: eu gostei do computador
Words list without stop words: ['gostei', 'computador']
```

Figura 15: Exemplo de remoção de palavras de parada que afeta o sentido do conjunto de palavras.

7 Expansão de Siglas, Abreviaturas e Contrações

Contrações, abreviaturas, siglas, etc., podem representar dificuldade ao sistema de PLN caso não sejam devidamente tratadas. Uma contração representa a supressão de uma ou mais letras na escrita e que pode significar alteração da pronúncia. Uma abreviatura corresponde a uma escrita compacta, ou seja, uma forma reduzida relacionada a uma determinada palavra. Uma sigla é formada pelas letras iniciais ou partes de várias palavras.

Exemplos:

- Copo d'água = Copo de água. (contração d'água)
- Ltda = Limitada (abreviatura)
- PLN = Processamento de Linguagem Natural (sigla)

Tipicamente, abreviaturas, contrações e siglas são substituídas utilizando-se expansões para as formas das palavras originais. Uma maneira simples de executar esse processo é através da utilização de dicionários de mapeamento.

Exemplo:

{“Ltda”: “limitada”, “pln”: “processamento de linguagem natural”, “d’água”: “de água”}

A Figura 16 ilustra esquematicamente a entrada e a saída do processo de expansão de siglas, contrações ou abreviaturas e a Figura 17 mostra um exemplo de entrada e saída do processo utilizando um dicionário para a execução dos mapeamentos.



Figura 16: Processo de expansão de siglas, abreviaturas e contrações.

Input text: O senhor joão deixou de trabalhar na Sistemas LTDA para ficar o dia todo sem fazer nda, só dorme e bebe um copo d'água a cada três horas

Output text: O senhor joão deixou de trabalhar na Sistema s Limitada para ficar o dia todo sem fazer nada, só dorme e bebe um copo de água a cada três horas

Figura 17: Exemplo de entrada e saída do processo de expansão de siglas, abreviaturas, contrações, etc.

8 Correção de Erros Gramaticais (Grafia de Palavras - *Spell Checker*)

A etapa de correção de erros gramaticais tem como objetivo a substituição de palavras que foram incorretamente escritas levando-se em conta a ortografia considerada correta pela norma culta. Essa etapa de pré-processamento é ilustrada esquematicamente na Figura 18.

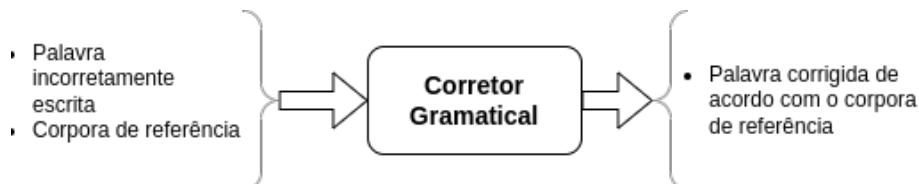


Figura 18: Etapa de correção de erros ortográficos.

Algumas vezes, palavras podem ser escritas de modo incorreto para expressar uma intensidade ou um sentimento. Outras vezes podem ser escritas de modo incorreto por mero engano/descuido.

Exemplo: Faz muito friiiiiiiiiiiiiioooooooooooooo.

No exemplo anterior, a palavra *friiiiiiiiiiiiooooooooooooo* foi escrita dessa forma para expressar uma sensação experimentada pelo autor do texto ou para refletir tal sensação para o leitor.

Para o caso de padrões com letras repetidas, uma alternativa é proceder com a sucessiva remoção destas, seguida de uma busca em um corpus de texto que pode ser utilizado como critério de parada.

Para um caso mais geral de grafia incorreta em que potencialmente podem ocorrer múltiplas correspondências, uma alternativa é a utilização de uma medida de distância entre a palavra incorreta e outras candidatas, obtidas por alterações sistemáticas aplicadas na palavra original.

Exemplo: pota

Substituições candidatas:

porta, pata, pote, portal

A substituição correta tipicamente leva em consideração uma análise da proximidade dos padrões de substituições em relação ao texto original e, também, a ocorrência no corpus de texto utilizado como referência.

Exemplo: Análise de proximidade (edit distance), considerando a palavra pota:

Porta → uma única operação de inserção de um *r* entre o e t.

Portal → duas operações de inserção, um *r* entre o e t e um *l* ao final da palavra.

O algoritmo de correção deve levar em consideração a probabilidade de ocorrência das substituições candidatas do corpus de texto tomado como referência para efetuar a correção.

O diagrama de atividades da Figura 19 modela em alto nível o processo de correção baseado em distância de edição e estatística de corpora de referência.

A Figura 20 ilustra um exemplo de correção ortográfica de palavra utilizando-se o Mac-Morpho e geração de palavras a uma edit distance de 0 e de 1.

Além dessa estratégia, bibliotecas tais como o *pyspellchecker* do Python podem ser utilizadas para implementar essa atividade na pipeline de pré-processamento.

9 Stemização

É o processo de redução de uma palavra à sua forma raiz sem que haja, no entanto, a preservação do significado, ou seja, o stem não necessariamente faz parte do léxico da linguagem.

Para esse processo a entrada é uma palavra e a saída é um "corte" ou stem da palavra, eliminando-se os afixos (prefixos e/ou sufixos). Esse processo é ilustrado esquematicamente na Figura 21.

Exemplo: aborrecida ← aborrec

Nesse caso, houve a eliminação do sufixo (característica do particípio) que foi agregado durante o processo de inflexão.

Tanto o processo de stemização como o processo de lematização, que será visto mais adiante, tem como objetivo reduzir os tamanhos dos vetores representativos das sentenças a serem utilizados nos sistemas de PLN.

Cabe ressaltar que o processo de stemização, assim como o de lematização, pode fazer com que palavras que remetam a um contexto diferente se tornem menos discriminativas/relevantes ao modelo de

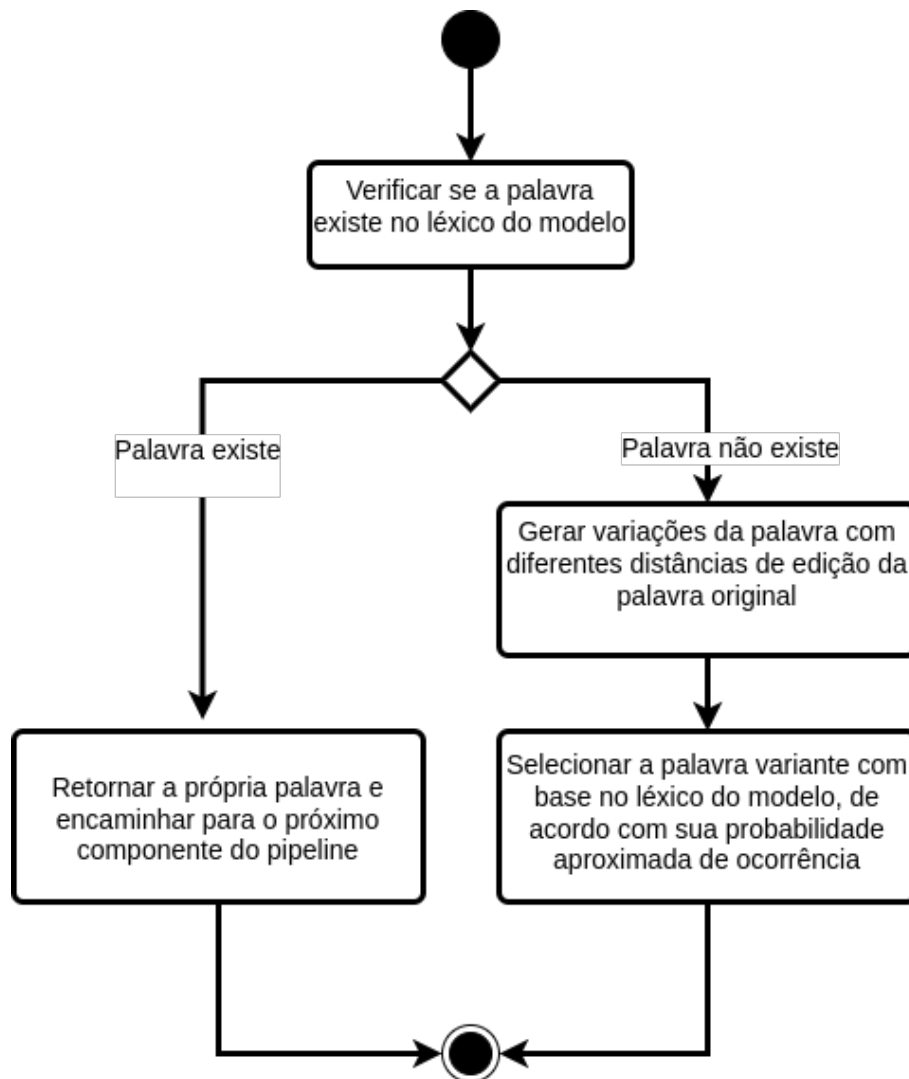


Figura 19: Correção de erros ortográficos utilizando estatísticas do corpora de referência.

```

Misspelled word: amenisar

Mac-Morpho word tokens after excluding non-word strings: ['jersei', 'atinge', 'm', 'dia', 'de', 'cr', 'milh', 'o', 'em', 'a', 'venda', 'de', 'a', 'pinhal', 'em', 's', 'o', 'paulo', 'programe', 'sua', 'viagem', 'a', 'a', 'exposi', 'o', 'nacional', 'do', 'zebu', 'que', 'come']

Generating 1-edit distance words from the original misspelled word:
Words that are 1 edit distance away from the original word: ['vmenisar', 'amenisao', 'dmenisar', 'imenisar', 'amsnizar', 'ameneisar', 'kmenisar', 'amenvisar', 'amenisaa', 'amtenisar', 'amenxisar', 'ahmenisar', 'ramenisar', 'vamenisar', 'hamenisar', 'amenisag', 'amonisar', 'amenisajr', 'amenosar', 'amenisarz', 'amenisav', 'amneisar', 'amenioar', 'amenirar', 'ymenisar', 'amenhsar', 'amenisary', 'amemisar', 'amenislr', 'amensiar']

-----
Verifying which belong to text corpus:
{'amenizar'}

Selecting one which is most likely right:
amenizar
  
```

Figura 20: Exemplo de correção ortográfica com estatística de corpus e edit distance.

aprendizagem de máquina de PLN.

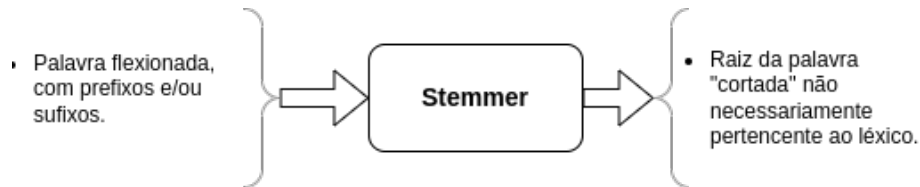


Figura 21: Processo de obtenção do stem de uma palavra

10 Lematização

O processo de Lematização é similar ao processo de stemização, mas, nesse caso, a palavra reduzida precisa fazer parte do léxico da linguagem. Dessa forma, pode-se concluir que a lematização é um processo muito mais aderente à estrutura da linguagem do que a stemização e, portanto, regras de transformação mais sofisticadas e que preservem algum conteúdo linguístico devem ser utilizadas.

A Figura 22 ilustra esquematicamente o processo de lematização em termos de entradas e saídas.

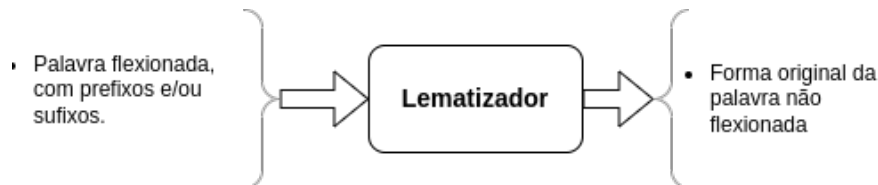


Figura 22: Processo de lematização

Exemplos:

aborrecida → aborrecer

florada → flor

No processo de lematização, é comum que uma determinada classe gramatical seja reduzida a outra de mesma categoria durante o processo (e.g.: verbo flexionado verbo no infinitivo, etc).

11 Rotulação - *Tagging*

A rotulação, conhecida como tagging, é a etapa de pré-processamento que identifica categorias de palavras presentes em uma frase de acordo com o contexto local. Nesse caso, contexto local refere-se a palavras próximas e que devem ser consideradas em conjunto ao se definir um determinado rótulo de palavra. A Figura 23 ilustra a entrada e a saída do processo de rotulação.

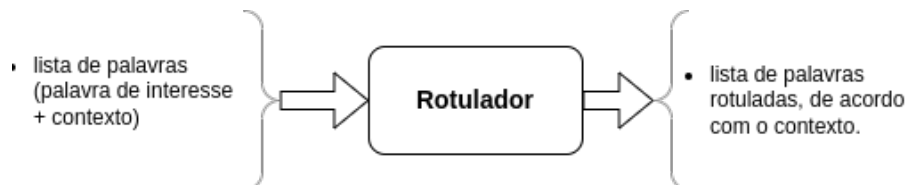


Figura 23: Processo de rotulação.

A rotulação torna possível o estabelecimento de relações entre as diferentes palavras, substituição por palavras equivalentes, cálculos de estatísticas de frases específicas, entre outras.

Exemplo: O carro está sujo.

Saída do processo de tagging: o: artigo (DET), carro: substantivo (NN), está: verbo (VB), sujo: adjetivo (ADJ)

A Figura 24 ilustra um exemplo de saída do processo de rotulação utilizando-se o modelo de rotulação do spaCy.

```
Original document: Hoje é um excelente dia para aprender PLN. Meus resultados depende  
m da minha dedicação!
```

```
Token text: ( Hoje ), Token lemma: ( hoje ), Token POS TAG: ( ADV )  
Token text: ( é ), Token lemma: ( ser ), Token POS TAG: ( AUX )  
Token text: ( um ), Token lemma: ( um ), Token POS TAG: ( DET )  
Token text: ( excelente ), Token lemma: ( excelente ), Token POS TAG: ( ADJ )  
Token text: ( dia ), Token lemma: ( dia ), Token POS TAG: ( NOUN )  
Token text: ( para ), Token lemma: ( para ), Token POS TAG: ( CONJ )  
Token text: ( aprender ), Token lemma: ( aprender ), Token POS TAG: ( VERB )  
Token text: ( PLN ), Token lemma: ( PLN ), Token POS TAG: ( PROPN )  
Token text: ( . ), Token lemma: ( . ), Token POS TAG: ( PUNCT )  
Token text: ( Meus ), Token lemma: ( meus ), Token POS TAG: ( DET )  
Token text: ( resultados ), Token lemma: ( resultado ), Token POS TAG: ( NOUN )  
Token text: ( dependem ), Token lemma: ( depender ), Token POS TAG: ( VERB )  
Token text: ( da ), Token lemma: ( de o ), Token POS TAG: ( ADP )  
Token text: ( minha ), Token lemma: ( meu ), Token POS TAG: ( DET )  
Token text: ( dedicação ), Token lemma: ( dedicação ), Token POS TAG: ( NOUN )  
Token text: ( ! ), Token lemma: ( ! ), Token POS TAG: ( PUNCT )
```

Figura 24: Processo de rotulação.

12 Normalização de Caixa

Normalização de caixa diz respeito à utilização de caracteres todos em caixa alta ou caixa baixa. Esse processo é utilizado primordialmente para facilitar o processo de comparação e casamento de padrões de um ou mais padrões específicos. O modelo em alto nível desse processo é ilustrado na Figura 25

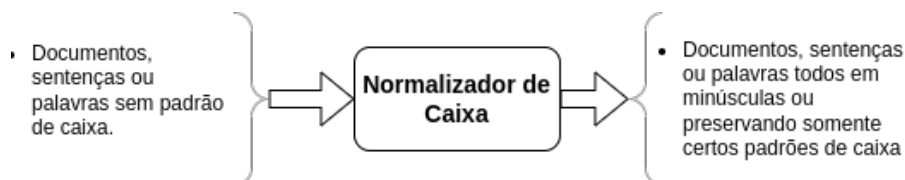


Figura 25: Processo de lematização

Assim como no caso da remoção de caracteres especiais, textos providos de caracteres capitalizados podem auxiliar o processo de tokenização de sentenças e palavras. Outro ponto relevante é que muitas vezes a utilização de caracteres capitalizados pode ter relação com nomes próprios, siglas e/ou abreviaturas, necessitando, portanto, de maior cuidado ao serem manipulados na pipeline (e.g., ANA pode ser um nome próprio ou uma sigla para Agência Nacional de Águas). Conclui-se, portanto, que essa etapa deve ser estrategicamente posicionada no fluxo de pré-processamento de modo que o comportamento geral da pipeline não seja prejudicado.

13 Análise Sintática - *Parsing*

O processo de análise sintática extrai de uma frase (sentença) suas principais partes, ou seja, o sujeito, o predicado, o verbo, etc. Esse processo é esquematizado de modo simplificado na Figura 26



Figura 26: Processo de parsing

Uma das utilizações do processo de parsing é identificar os agentes (quem efetua as ações), o paciente (quem sofre as ações) e as ações propriamente ditas.

A Figura 27 ilustra um conjunto de frases de entrada e a saída do processo de análise sintática usando hierarquia. Esse exemplo foi construído utilizando-se a biblioteca spaCy.

```
Original reviews: ['O produto chegou quebrado', 'A entrega foi feita com atraso.', 'Os valores das ações despencaram no último pregão.', 'Eu não gostei do computador.', 'A entrega foi feita no prazo.']

[{'sentence': 'O produto chegou quebrado', 'subject': 'O produto', 'verb': 'chegou', 'object_predicate': 'quebrado'}]
[{'sentence': 'A entrega foi feita com atraso.', 'subject': 'A entrega', 'verb': 'feita', 'object_predicate': 'com atraso'}]
[{'sentence': 'Os valores das ações despencaram no último pregão.', 'subject': 'Os valores das ações', 'verb': 'despencaram', 'object_predicate': 'no último pregão'}]
[{'sentence': 'Eu não gostei do computador.', 'subject': 'Eu', 'verb': 'gostei', 'object_predicate': 'do computador'}]
[{'sentence': 'A entrega foi feita no prazo.', 'subject': 'A entrega', 'verb': 'feita', 'object_predicate': 'no prazo'}]
```

Figura 27: Processo de parsing

Para maiores detalhes e aprofundamentos, o leitor deve buscar referências tais como como (AGGARWAL, 2023), (SARKAR, 2019) e (FERREIRA; LOPES, 2025)

14 Exercícios e Problemas

OBSERVAÇÃO: Na solução dos exercícios e problemas seguintes, prepare uma pequena apresentação explorando os conceitos envolvidos no problema/exercício, justificando e explicando em detalhes sua resposta. Além disso, caso o problema requeira o desenvolvimento de um programa computacional e não tenha sido mencionada nenhuma linguagem de programação, utilize a linguagem de programação Python 3.* e forneça como resposta o código-fonte, o procedimento de execução e o arquivo referente às dependências (e.g., requirements.txt, package.json, etc). Caso o enunciado trate de um pequeno exemplo de código executável e não de um sistema, pode ser utilizada a tecnologia do Jupyter Notebook.

- TC.1. Discuta sobre questões de codificação de texto para armazenamento em banco de dados. Forneça um exemplo executável, utilizando um banco de dados de sua preferência, em que o uso incorreto de codificação gera perda de conteúdo comunicativo (**Hint:** Considere, por exemplo, o armazenamento de uma conversa de whatsapp, contendo um emoji, em um banco de dados.)
- TC.2. Faça um breve estudo comparativo sobre como metacaracteres e expressões de texto são utilizadas em expressões regulares nas linguagens de programação Python e JavaScript. Na sua discussão inclua necessariamente os exemplos de metacaracteres mostrados nesse texto além de mais outros 5 que não tenham sido discutidos em aula. Forneça pelo menos 5 exemplos que incluam a aplicação de regex para encontrar um determinado texto de interesse ou então efetuar tokenização por sentenças.
- TC.3. Cite exemplos de tokens de frases e tokens de palavras que podem significar:
- Uma opinião negativa referente a um produto de uma loja de vestuário.
 - Uma opinião positiva referente a um produto de uma loja de vestuário.
 - Uma opinião neutra referente a um produto de uma loja de vestuário.
 - Uma opinião negativa relacionada a um carro vendido por uma concessionária automotiva.
 - Uma opinião positiva relacionada a um carro vendido por uma concessionária automotiva.
 - Uma opinião neutra relacionada a um carro vendido por uma concessionária automotiva
- TC.4. Um psicólogo deseja recomendar uma obra literária para entreter seu paciente que está com problemas de depressão. Esse psicólogo sabe que certas palavras podem contribuir para motivar o leitor e melhorar seu estado geral. Explique como uma análise baseada em estatística de tokens de palavras pode ser aplicada a um conjunto de obras que o psicólogo pode recomendar aos seus pacientes objetivando atingir êxito em seu tratamento. Para corroborar sua resposta, dê um exemplo simples a partir de um texto publicamente disponível e realize a análise dos tokens de palavras desse documento (utilize um capítulo ou trecho contendo no mínimo 5000 palavras.). Diga como o mesmo tipo de análise pode ajudar o psicólogo a não recomendar leituras que acarretem a obtenção de efeitos contrários. Ilustre também com um exemplo.

- TC.5. Faça um estudo comparativo entre o Snowball Stemmer e o Stemmer RSLP. No seu estudo descreva as regras básicas utilizadas para efetuar os cortes nas palavras e compare o desempenho do stemizador considerando diferentes classes gramaticais. Diga, respaldado por um exemplo computacional simples, em qual situação cada um deles pode apresentar melhor desempenho.
- PC.1. Desenvolva um programa simples que converta 10 emojis comumente utilizados em conversas de chat em texto que reflita a ideia geral do emoji. A entrada para seu programa deve ser um texto escrito com codificação utf-8, contendo emojis, e a saída deve ser o texto com a conversão solicitada.
- PC.2. Desenvolva um programa que leia uma sequência de bytes e que mostre que diferentes caracteres quando codificados utilizando o padrão UTF-8 ocupam uma quantidade diferente de bytes. A entrada para seu programa deve ser a sequência de byte ou um arquivo de texto codificado em utf-8 e a saída deve ser o caractere unicode correspondente e o número de bytes utilizados na representação de cada caractere. (OBS: Leia os bytes do arquivo e não os caracteres).
- PC.3. Crie uma conversa de whatsapp contendo emojis. Exporte para um arquivo codificado usando o utf-8. Desenvolva um programa que leia o arquivo da conversa e que faça a tradução dos emojis para expressões textuais apropriadas.
- PC.4. Demonstre a extração de dados a partir de uma página de Internet que contenha uma das seguintes opções de conteúdo:
- (a) Blog pessoal (não pode ser o seu).
 - (b) Notícias jornalísticas
 - (c) Revisões ou descrições de um produto.
- Na sua resposta considere que a saída da extração é o componente de conteúdo que contém somente a informação de interesse, ou seja, ignore componentes da página que contenham, por exemplo, imagens, scripts, etc, mas sem desconsiderar, por exemplo, marcações de parágrafos.
- PC.5. Desenvolva um componente de pré-processamento que faça uso de expressão regular e que recupere o conteúdo textual de interesse a partir de uma página de notícias, ou blog pessoal ou, ainda, um site de e-commerce. Ou seja, seu componente deve ser testado sobre a resposta a uma requisição HTTP/HTTPS e deve extrair o conteúdo de interesse de acordo com o conteúdo do site/sistema, eliminando todo o ruído presente em tags que não possuem conteúdo linguístico relevante.
- PC.6. Desenvolva um pequeno componente para tokenização por sentenças que utilize regex. Demonstre o funcionamento do seu tokenizador a partir da rapagem de dados de uma página da Internet, tal como um blog, página de notícias, site de e-commerce, etc.
- PC.7. Discuta sobre diferentes modelos de tokenizadores de sentença disponíveis em bibliotecas tais como NLTK e Spacy. Na sua discussão, faça uma comparação envolvendo o número de tokens gerados considerando uma obra de domínio público, escrita em português e disponível no projeto Gutenberg.

- PC.8. Desenvolva um pequeno componente para tokenização por palavras que utilize somente regex. Teste o funcionamento do seu componente utilizando algum conteúdo textual de domínio público, tal como uma obra literária disponível no projeto Gutenberg. Obs: Para a tokenização por sentenças você pode utilizar um componente pré-treinado disponível em alguma biblioteca tal como NLTK.
- PC.9. Desenvolva uma pequena pipeline de tokenização por palavras que faça uso de dois processos de tokenização. Na primeira etapa você deve usar o tokenizador de sentenças do NLTK. Na segunda etapa você deve eliminar os tokens que não são palavras utilizando expressões regulares. Contabilize o número de tokens antes e depois do uso de regex e utilize como entrada um arquivo de texto, contendo pelo menos 5000 palavras, obtido a partir de texto disponível do projeto Gutenberg.
- PC.10. Repita o Problema PC.9. mas utilizando o tokenizador de palavras do spaCy na primeira parte da sua pipeline.
- PC.11. Desenvolva um analisador de sentimentos simples baseado em estatísticas de tokens de palavras. Seu analisador deve possuir como entrada um documento de texto (lido a partir de um arquivo .txt). A saída do analisador deve ser se a opinião manifestada no documento é positiva, negativa ou neutra, de acordo com a contagem simples do número de ocorrências de tokens de palavras consideradas como positivas ou negativas (quando houver empate a saída deve ser sentimento neutro). Seu analisador deve ser configurável em termos de quais palavras serão consideradas positivas ou negativas. Além disso, o analisador deve estimar a probabilidade de acerto da análise de sentimentos (e.g., se a contagem de palavras positivas for 3 e a contagem de palavras negativas for 1, o analisador deve dizer que a saída é "positiva com probabilidade de 75%")
- PC.12. Repita o Problema PC.11. mas considerando como unidade básica de análise uma sentença porém contabilizando os pontos das palavras contidas na sentença (e.g., se uma sentença possuir 3 palavras positivas e uma palavra negativa, então o score de sentenças soma 1 ao positivo, caso o número de sentenças positivas seja maior do que o número de sentenças negativas então o documento é considerado positivo). A saída do analisador deve ser o sentimento predominante do documento e a probabilidade estimada de que o sentimento de fato reflita o conteúdo analisado (e.g., "saída positiva com probabilidade de X%").
- PC.13. Desenvolva um programa que compare a saída de um corretor ortográfico baseado em um corpora, tal como o Mac-Morpho, e aquela obtida através da utilização de uma biblioteca tal como o *pyspellchecker*. Faça um estudo envolvendo a grafia incorreta de pelo menos 50 palavras, pertencentes a diferentes contextos (e.g., literatura técnica, receitas, linguagem coloquial, etc.) e explique os resultados obtidos.
- PC.14. Desenvolva um sistema do tipo chatbot que funcione como um corretor ortográfico para o usuário. Seu chatbot deve receber como entrada uma frase digitada pelo usuário e deve processá-la e fornecer como resposta o seguinte:

- Caso a frase esteja gramaticalmente correta, o chatbot deve responder: "Parabéns, sua frase está gramaticalmente correta".
- Caso a frase apresente palavras com grafias incorretas, o chatbot deve responder: "Será que você não quis dizer: ..." e reescrever a frase do usuário com as palavras corrigidas para suas versões mais prováveis utilizando critério de menor *edit distance* e as estatísticas do corpora de referência.

Para esse problema, desenvolva seu sistema com interface do usuário baseada em página da internet (formulário simples) e com o processamento do corretor inteiramente efetuado pelo *back end*.

PC.15. Conforme discutido ao longo desse trabalho, modelos de rotulação nem sempre estão totalmente ajustados com os rótulos mais adequados à linguagem. Dê um exemplo prático de fine-tuning para um modelo de tagging do spaCy.

PC.16. Desenvolva um pequeno chatbot que utilize o processamento do tipo dependency parsing e que seja capaz de identificar quem é o responsável por solicitar uma ação

PC.17. Aplique técnicas de tokenização e correção de erros de ortografia a dados de revisão de produtos que tenham sido raspados de uma página de revisão da Internet. Ilustre o comportamento e o desempenho do seu trecho de pipeline de PLN identificando os principais gargalos e sugira uma melhoria possível. Esclareça o porquê da ordem dos elementos em sua pipeline. Na sua resposta ilustre, através de uma aplicação web, desenvolvida em Python, com front end e back end, os seguintes pontos:

- O tempo necessário para se efetuar a raspagem de dados.
- O tempo necessário à remoção de ruído de marcação.
- O tempo necessário à quebra em tokens de frases e tokens de palavras.
- O tempo necessário à substituição de abreviaturas
- O tempo necessário para se processar e corrigir palavras digitadas incorretamente.

a) A entrada para seu programa deve ser uma URL. As saídas devem ser cada uma das etapas citadas da pipeline de pré-processamento e os tempos necessários para se executar tais etapas.

Etapa	Entrada	Saída	Duração (ms)
Extração	http://teste.com/	"<html>...textos</html>"	5ms
Remoção de Tags	"<html>...textos</html>"	"Produto interessante. ..."	1ms
Tokenização por frases	"Produto interessante..."	["Produto interessante", "Gostei do produto"]	2ms
...	...	...	...

Tabela 3: Exemplos de metacaracteres utilizados em regex do Python

Referências

AGGARWAL, C. C. *Machine Learning for Text*. Switzeland: Springer Nature Switzeland, 2023. ISBN 978-30-309-6622-5.

FERREIRA, M.; LOPES, M. *Para Conhecer Linguística Computacional*. São Paulo, SP: Contexto, 2025. ISBN 978-85-520-0152-2.

SARKAR, D. *Text Analytics with Python*. 2nd. ed. Berkeley, CA: Apress, 2019. ISBN 978-1-484-24353-4.